Escola Politècnica Superior d'Enginyeria de Vilanova i la Geltrú
EPSEVG

Enunciat Laboratori 1 ESC2

Entrada/Sortida per Enquesta i tractament de bits

UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

PRÀCTICA 1:

E/S per enquesta i tractament de bits

Introducció

A continuació es descriuen els registres dels principals dispositius que s'utilitzaran en aquesta pràctica, i el seu funcionament. En aquells registres que són exclusivament d'escriptura, els bits que no tenen un significat concret s'indiquen com "indefinit", i vol dir que no importa quin valor s'hi escrigui, ja que el dispositiu ignora aquests bits. A més, el valor d'aquests registres resta indefinit un cop finalitzada la transferència, excepte per als registres ***Rfil_pant***, ***Rcol_pant*** i ***Rdat_pant***, els quals són "persistents", és a dir que conserven l'últim valor que s'hi ha escrit.

Pantalla

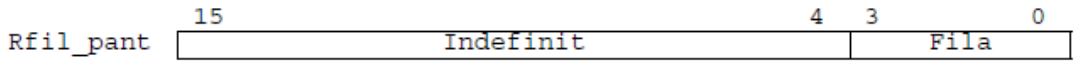
La pantalla és un display rectangular de 16 línies per 64 columnes. Els caràcters que s'hi escriuen poden usar sols blanc o negre per al primer pla i per al fons, adoptant dues apariències, anomenades modes o atributs: mode vídeo normal (mateix fons que la resta de pantalla) i mode vídeo invers (fons del caràcter diferent que la resta de la pantalla). Aquest dispositiu és únicament de sortida, és a dir, no es poden llegir caràcters que hi hagin estat escrits.

La pantalla **no necessita sincronització** ja que és un dispositiu ràpid. Es considera que quan acaba la instrucció que dóna l'ordre d'escriptura en pantalla, el caràcter ja s'ha escrit. Aquest dispositu es controla amb 4 registres, associats als ports 0x04, 0x05, 0x06 i 0x07:

Rfil_pant:

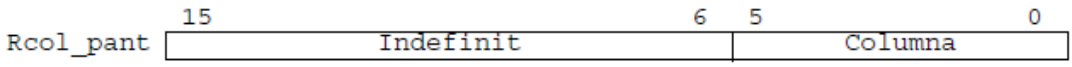
Registre d'escriptura. Indiquem la fila on volem escriure un caràcter (4 bits).

Aquest registre està associat al port 0x05 (***Rfil_pant*** al fitxer crt0.s). El seu format és:



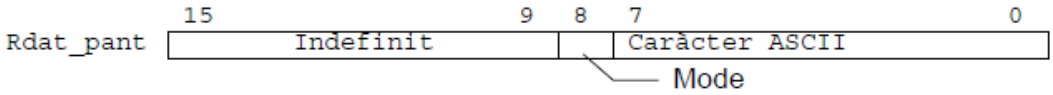
Rcol_pant:

Registre d'escriptura. Indiquem la columna on volem escriure un carácter (6 bits). Està associat al port 0x06 (***Rcol_pant***). El seu format és:

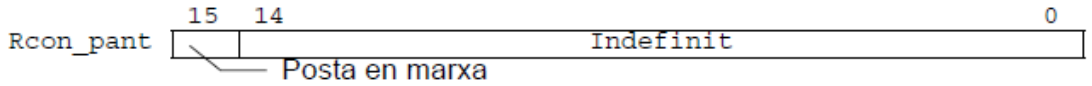


Rdat_pant:

Registre d'escriptura. Indiquem el codi ASCII del caràcter (8 bits) i el mode de vídeo en què ha d'aparèixer (1 bit). Si el bit de mode val 0, s'escriu en mode normal; si val 1, en mode invers. Està associat al port 0x07 (***Rdat_pant***). El seu format és:



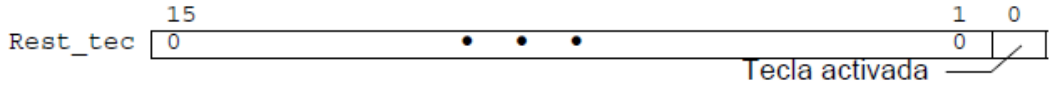
Rcon_pant:
Registre d'escriptura. L'escriptura d'un caràcter en pantalla pot requerir accedir a un o més registres, però no es fa realment efectiva fins que escrivim un 1 en el **bit de posta en marxa** d'aquest registre. Quan ho fem, el darrer caràcter (i mode) escrit al registre Rdat_pant es mostrarà per pantalla, a la darrera fila i columna escrites en Rfil_pant i Rcol_pant respectivament. Està associat al port 0x04 (***Rcon_pant***). El seu format és:



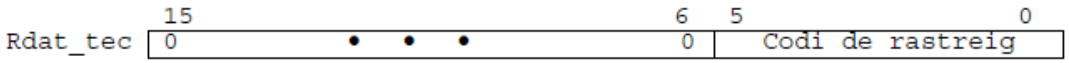
Teclat

El teclat consta de 43 tecles totes elles situades al bloc principal del teclat. Inclou les tecles marcades amb les lletres de la A a la Z (excepte la Ñ i la Ç); amb els números del 0 al 9; amb els signes de puntuació coma, punt, guió i suma (, . - +); i les tecles ESPAI, INTRO i BACKSPACE. Cadascuna d'aquestes tecles està associada a un identificador (entre el 0 i el 42) que anomenem ***codi de rastreig***. El teclat es controla amb 3 registres, associats als ports 0x08, 0x09 i 0x0A.

Rest_tec:
És un registre de lectura. El bit de tecla activada es posa a 1 quan es polsa alguna tecla, i es manté així encara que la alliberem. El bit es torna a posar a 0 automàticament quan una instrucció llegeix el registre ***Rdat_tec***. Està associat al port 0x0A (àlies ***Rest_tec***). El seu format és:



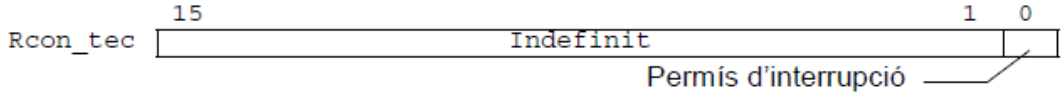
Rdat_tec:
És un registre de lectura, d'on es pot llegir el codi de rastreig (6 bits) que identifica la tecla polsada. Quan es llegeix aquest registre, automàticament s'indica al teclat que aquest codi s'ha llegit i aquest posa a 0 el bit de tecla activada del registre Rest_tec. El resultat de llegir Rdat_tec estant el bit de tecla activada a 0 és indefinit (p.ex., en la segona de dues lectures seguides!). Està associat al port 0x09 (àlies ***Rdat_tec***) i el seu format és:



A més a més, en el fitxer crt0.s s'ha definit globalment un vector de 43 caràcters anomenat ***tteclat***. Conté els caràcters que associarem a cada tecla o codi de rastreig, de forma que el codi ASCII s'obté simplement llegint ***tteclat[codi de rastreig]***.

```
tteclat:    .ascii "1234567890\bQWERTYUIOP+ASDFGHJKL\nZXCVBNM,.- "
            .balign 2
```

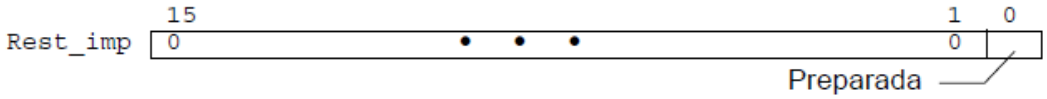
Rcon_tec:
És un registre d'escriptura, i li indiquem al teclat si s'ha de sincronitzar per interrupcions (permís d'interrupcions = 1) o per enquesta (permís = 0). Està associat al port 0x08 (àlies Rcon_tec) i el seu format és:



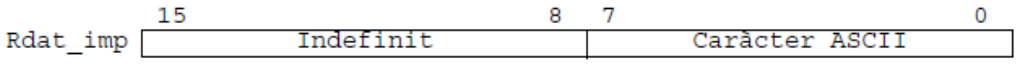
Impressora

La impressora permet imprimir línies de 64 caràcters sobre una cinta de paper. El funcionament de la impressora és similar al del teclat. El controlador d'impressora té 3 registres:

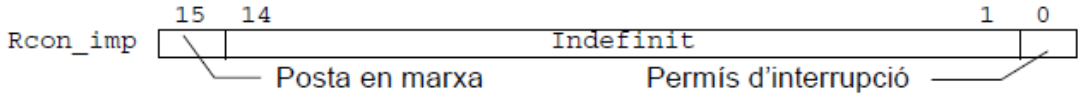
Rest_imp:
És un registre de lectura, i indica l'estat de la impressora. El bit de "preparada" indica si la impressora ha acabat la feina i està llesta per a rebre caràcters (preparada = 1) o està ocupada imprimint (preparada = 0). En el primer cas, la finestra mostra el missatge "***Ready***" i en el segon cas el missatge "***Busy***". No es pot predir el temps que tardarà la impressora en imprimir un caràcter, entre altres raons perquè es pot produir un error del dispositiu (manca de paper o tinta, o un embús, o posta fora de línia), que retardi l'escriptura fins que l'usuari resolgui el problema manualment. El depurador **sisa-dbg** simula aquests errors generant-los de forma aleatòria pero amb una freqüència molt baixa, de fet és molt poc probable que t'hi trobis. En cas d'error la finestra de la impressora mostra el missatge "*Out of Paper*" i el registre d'estat té el bit de preparada a zero. La impressora no retorna a l'estat de "preparada" fins que l'usuari "subsani" manualment el problema, enfocant aquesta finestra (amb TAB) i polsant la tecla d'espai. Aquest registre està associat amb el port 0x3A (àlies ***Rest_imp***). El seu format és:



Rdat_imp:
És un registre d'escriptura, on li indiquem el caràcter ASCII que volem enviar a la impressora (8 bits). Està associat al port 0x39 (àlies ***Rdat_imp***). El seu format és:



Rcon_imp:
És un registre d'escriptura, i permet indicar a la impressora si se sincronitzarà per interrrupcions (1) o per enquesta (0). A més, escrivint un 1 al **bit de posta en marxa** li indiquem que ha de començar a imprimir el caràcter emmagatzemat al registre de dades (*Rdat_imp*). Tanmateix, el comportament de la impressora és impredecible si es fa la posta en marxa sense assegurar-se que està a 1 el bit de preparada. Està associat amb el port 0x38 (àlies ***Rcon_imp***). El seu format és:



Estat Inicial dels dispositius

Quan el SISP-F s'inicia, tots els dispositius estan preparats per sincronitzar-se per enquesta, ja que els respectius bits de permís d'interrupcions estan a 0, així com el bit I del registre *PSW* del processador (bit 1 de S7). A més a més, la impressora està inicialment preparada.

Tractament de bits

Desplaçaments de bits

Per desplaçar els bits d'un número una posició a la dreta o a l'esquerra es copia cada bit a la posició adjacent. En un desplaçament de múltiples posicions, el resultat és el mateix que s'obtindria de desplaçar un lloc múltiples vegades.

Desplaçar els bits d'un número una posició a la dreta es pot fer de dues formes diferents: **desplaçament lògic** i **desplaçament aritmètic**. Mentre que en un desplaçament lògic a la dreta, el bit de major pes passa a valdre 0, en un desplaçament aritmètic a la dreta, aquest bit no es modifica. Per als desplaçaments d'una posició a l'esquerra no hi ha diferència entre lògic i aritmètic, i el bit de menor pes sempre passa a valdre 0.

El repertori SISA-F disposa de les instrucions ***SHL*** i ***SHA*** per als desplaçaments lògics i aritmètics de k posicions, respectivament. Valors positius de k indiquen desplaçaments a l'esquerra, valors negatius a la dreta.

Exemples:

```
MOVI Rb, 3
SHL Rd, Ra, Rb        ; Rd <- Ra despl. lògic a l'esquerra 3 bits
                      ; En desplaçaments a l'esquerra SHL == SHA

MOVI Rb, -3
SHL Rd, Ra, Rb        ; Rd <- Ra despl. lògic a la dreta 3 bits
MOVI Rb, -3
SHA Rd, Ra, Rb        ; Rd <- Ra despl. aritmètic a dreta 3 bits
                      ; Diferent de SHL ja que extén el signe
```

Un dels usos freqüents dels desplaçaments és el de multiplicar i dividir per potències de 2. El desplaçament d'un lloc a l'esquerra d'un nombre natural o enter equival a multiplicarlo per 2. Tant el desplaçament lògic d'un lloc a la dreta d'un nombre natural com el desplaçament aritmètic d'un lloc a la dreta d'un nombre enter parell o positiu equival a dividir el nombre per 2. En canvi, per als enters senars negatius l'operació de desplaçament a la dreta no equival a la divisió per 2, com la que fa la instrucció DIV, on el reste té sempre el signe del dividend, sinó que obté un quocient i reste diferents, on el reste és sempre positiu.

Operacions lògiques bit a bit

Les operacions lògiques ***AND*** i ***OR*** bit a bit s'usen freqüentement per modificar bits individuals dintre de una dada. Un operand conté la dada a modificar i l'altre operand conté un patró de bits, anomenat *màscara*, que indica quins bits es modificaran i quins no. L'operació AND serveix per posar bits a zero, i la màscara corresponent ha de tenir

a zero solament aquells bits a modificar. A la inversa, l'operació OR serveix per posar bits a u, i la màscara corresponent ha de tenir a u solament aquells bits a modificar. El repertori d'instruccions SISA-F disposa de les instrucions AND i OR bit a bit. Per exemple:

```
$MOVEI R2, 0b1111111111110011
AND R1, R1, R2                  ; posa a 0 els bits 2 i 3 de R1

$MOVEI R2, 0b0000000000001100
OR R3, R3, R2                   ; posa a 1 els bits 2 i 3 de R3
```

També està disponible la instrucció NOT bit a bit. En l'anterior exemple, hauríem pogut substituir la tercera línia (macro $MOVEI) per una NOT, obtenint idèntic resultat:

```
$MOVEI R2, 0b1111111111110011
AND R1, R1, R2
NOT R2, R2
OR R3, R3, R2
```

La darrera instrucció bit a bit és el XOR, que ens permetrà complementar els bits que tinguin un 1 a la màscara:

```
$MOVEI R1, 0x0F0F
$MOVEI R2, 0x00FF
XOR R1, R1, R2                  ; R1 = 0x0FF0
```

ACTIVITATS A REALITZAR:

Les activitats d'aquesta sessió treballen amb els dispositius d'entrada/sortida. No oblideu incloure al principi del programa el fitxer de definicions dels dispositius, amb la directiva:
.include "crt0.s"

Per observar els dispositius d'entrada/sortida en el simulador cal activar el mode "***emulador***", amb la **tecla F2**. En aquest mode es mostren 3 panells. A dalt a l'esquerra, la sortida per pantalla del SISP-F, en format de 16 files per 64 columnes. A baix, el que surt per la impresora (s'escriu en la última fila, i es va desplaçant amunt a mesura que hi escrivim línies). A dalt a la dreta, el contingut actual dels registres d'estat de tots els dispositius, així com dels registres "persistents" de la pantalla i del disc (*Rfil_pant, Rcol_pant, Rdat_pant, Rcara_disc, Rpist_disc, Rsect_disc* i *Radr_disc*). En el mode "*emulador*" estan disponibles la majoria de les comandes relatives a l'execució contínua o pas a pas. Si es desitja observar el contingut dels registres o la memòria, sols cal passar novament al mode "***depurador***" amb la mateixa **tecla F2**. Podem passar d'un mode a l'altre tants cops com calgui.

Activitat 4.A: La pantalla

En aquesta activitat accedirem a la pantalla. Completa l'exercici 4.1 abans de continuar.

Exercici 4.1: Escriu un programa en alt nivell que mostri una 'A' a la posició [4,8] de la pantalla, amb atribut invers, i una 'B' a la posició [4,9] amb atribut normal. Observa que la fila és la mateixa en els dos casos, així que sols cal escriure-la un cop.

Tradueix el codi anterior a assemblador SISA-F, en el fitxer **s4a.s**. Assembla'l, i verifica amb el simulador que funcioni correctament.

Activitat 4.B: El teclat

L'objectiu d'aquest apartat és provar l'accés al teclat sincronitzant-lo per enquesta. Completa l'exercici 4.2 abans de continuar:

Exercici 4.2: Escriu un programa en alt nivell que esperi fins que es polsi una tecla qualsevol, i llavors escrigui una 'A' a la posició [4,8] de la pantalla, amb atribut normal.

A continuació tradueix el codi de l'exercici 4.2 a assemblador SISA-F, en el fitxer **s4b.s** i verifica'n el correcte funcionament amb el simulador.

Un cop funcioni l'anterior programa, completa l'exercici 4.3:

Exercici 4.3: Escriu una nova versió del programa en alt nivell anterior per tal que, en comptes d'una 'A', escrigui el caràcter associat a la tecla polsada. Fixa't que es tracta de fer sols un petit canvi en la finalització. Recorda que per traduir el codi de rastreig del teclat es disposa del vector tteclat.
Ara, tradueix l'anterior codi a assemblador SISA-F en el mateix fitxer **s4b.s** (sols cal afegir les modificacions). Assembla'l i comprova que funciona, amb el simulador.

Un cop comprovat el programa anterior, completa l'exercici 4.4:

Exercici 4.4: Escriu en alt nivell una nova versió del programa anterior, tal que faci la mateixa tasca (llegir una tecla i escriure-la en pantalla) però repetidament, fins que la tecla polsada sigui una 'F'. Fixa't que es tracta sols d'insertar l'anterior programa dins un bucle.

Tradueix el programa anterior a assemblador SISA-F en el mateix fitxer **s4b.s** (afegint sols les modificacions). Verifica'l amb el simulador.

Activitat 4.C La impresora

L'objectiu d'aquest apartat és provar l'accés a la impressora sincronitzant-se per enquesta.
Completa l'exercici 4.5 abans de continuar:

Exercici 4.5: Fes un programa en alt nivell que imprimeixi la paraula "Fi". Abans d'imprimir cada caràcter cal esperar que estigui preparada. I al final del programa, també.

A continuació tradueix el codi de l'exercici 4.5 a assemblador SISA-F en el fitxer **s4c.s** i verifica que funcioni correctament, amb el simulador. Recorda que el simulador pot generar errors de la impressora de forma aleatòria (el panell corresponent mostrarà el missatge "Out of paper"), fent que la impressora es quedi ocupada indefinidament. Si passa això, tan sols has d'enfocar el panell d'impressora (tecla TAB) i polsar la tecla d'espai, i la impressora es posarà preparada novament. En acabat, no oblidis d'enfocar novament el panell de pantalla.

Un cop funcioni l'anterior programa, completa l'exercici 4.6:

Exercici 4.6:
Escriu en alt nivell una nova versió del programa anterior que escrigui el vector frase a la impressora. Fixa't que el vector és un *string*, per tant, acaba amb un byte a zero (valor binari 0). Es tracta doncs de fer un bucle que iteri fins a trobar el zero final.

```
char frase[30] = "Aquest programa funciona";
```

Tradueix el programa anterior a assemblador SISA-F en el mateix fitxer **s4c.s**. Assembla'l i comprova amb el simulador el seu funcionament.

Activitat 4.D

Visualització d'operacions de tractament de bits

L'objectiu d'aquest apartat és combinar l'accés a la pantalla, al teclat, i les operacions de tractament de bits, i ho farem en 3 parts:
En primer lloc, escriurem un programa que mostri a les posicions [0,0]-[0,15] de la pantalla els 16 bits de la representació en binari (uns i zeros) de la variable ***w*** de mida word (figura 4.1), mostrant el bit més significatiu a la columna 0 i el menys significatiu a la columna 15. El programa ***main*** fa una crida a la subrutina ***mostra*** passant-li com a paràmetre per valor la variable w. Aquesta subrutina no retorna res ja que és una acció, però mostra la representació binària del paràmetre i a les posicions [0,0]-[0,15]. Completa l'exercici 4.7 abans de seguir:

```
int w=0x8888;
main(){
      mostra(w);
}
void mostra(int i){
      register int col=16;
      out (Rfil_pant,0);
      do{
            col--;
            out(Rcol_pant,col);
            out(Rdat_pant,'0'+(i&0x1));
            out(Rcon_pant, 0x8000);
            i=i>>1;
      }while (col>0)
}
```

Figura 4.1. Codi en alt nivell del programa main i de la subrutina mostra

Exercici 4.7: Tradueix a SISA-F el programa main i la subrutina mostra de la figura 4.1

Escriu el codi anterior al fitxer **s4d.s,** i verifica que funcioni bé amb el simulador.

En segon lloc, volem extendre el programa anterior, perquè es comporti així:

1) Mostrar el valor de *w* (el codi del programa anterior).
2) Esperar fins que es polsi una tecla.
3) Modificar *w* segons quina sigui la tecla polsada:
   - Si la tecla és una 'A', fer un desplaçament **lògic** de *w* (SHL), 1 lloc a l'**esquerra**.
   - Si és una 'B', fer un desplaçament **lògic** de *w* (SHL), 1 lloc a la **dreta**.
   - Si és una 'C', fer un desplaçament **aritmètic** de *w* (SHA), 1 lloc a la **dreta**.
   - Si és una 'D', s'ha de **dividir per dos** l'enter *w* (DIV).
   - Si és un número *n* (tecles '0'.... '9'), s'han de **complementar** els *n* bits de menys pes de *w*. Nota: aquesta operació es pot escriure en C així: w = wˆ((1<<n)-1); on l'operador ˆ significa XOR bit a bit, i l'operador << significa desplaçament a Esquerra
4) Repetir des del pas número 1 (el programa no acaba, fa un bucle sense fi).

Completa l'exercici 4.8 abans de continuar:

Exercici 4.8: Escriu en alt nivell (en C) el nou programa principal (main), suposant que l'acció mostra és la que està declarada a la figura 4.1.

A continuació, tradueix el programa anterior a SISA-F, en el mateix fitxer **s4d.s**, substituint el codi del main que hi teníeu abans, per aquest. Assembla'l i executa'l.
Verifica que el programa funciona, per exemple amb un valor inicial de w = 0x8888. La primera línia de la pantalla hauria de mostrar la cadena "1000100010001000". Si es prem la tecla 'A', la primera línia de la pantalla hauria de mostrar la cadena "0001000100010000".
Ves provant a prémer totes les tecles habilitades ('A', 'B', 'C', 'D', '0', ... '9'), i verifica que realitzen la funció desitjada.

En tercer lloc volem provar el funcionament del programa amb un enter negatiu. Modifica la inicialització de w en el fitxer s4d.s per tal que w = 0xFFF9. Torna a assemblar i muntar el programa, i executa'l.

Exercici 4.9:
Comprova que quan w és negatiu (w = 0xFFF9), la divisió DIV per 2 (tecla 'D') dóna un resultat diferent que el desplaçament SHA una posició a la dreta (tecla 'C'). Per què?